Summary of “PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU”

Tanush Mittal (tmittal), Satyam Goyal (sagoyal), Arjun Laxman (arlx), Kushal Patel (patelku)

Problem and Motivation

Recent interest has grown in enabling large language model (LLM) inference on consumer-grade GPUs. This would improve privacy, accessibility, and flexibility for users by allowing models to run locally rather than relying on cloud-based systems.

However, consumer GPUs are not optimized for LLM inference. They are designed primarily for graphics or mixed workloads, leading to inefficiencies such as excessive reliance on FP32 computation and limited memory and bandwidth compared to data center GPUs.

Also, modern GPU architectures employ a memory hierarchy optimized for data locality, which works well for training but is suboptimal for inference. Inference requires loading and accessing nearly all model weights, making memory access patterns far less cache-friendly. For example, the NVIDIA RTX 4090—a high-end consumer GPU costing around $2,000—has significantly less memory capacity and bandwidth compared to an NVIDIA A100, which costs roughly $20,000. This disparity makes it challenging to run large models on consumer hardware without performance degradation.

Related Works

Existing approaches to enabling LLM inference on limited-memory hardware primarily rely on offloading techniques. These methods aim to balance GPU memory usage and computation efficiency by strategically dividing work between GPU and CPU resources.

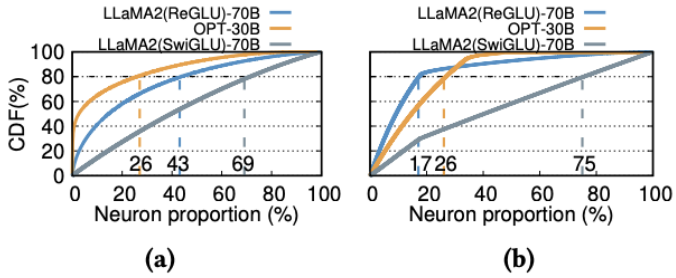
Layer-wise Offloading
One common approach is to divide computation across layers, offloading some of them to CPU memory when GPU memory is insufficient. FlexGen, for example, employs a “zigzag” scheduling strategy that prioritizes throughput by overlapping computation and data transfer. However, this method still spends a large portion of its execution time transferring weights between CPU and GPU, which severely limits performance.

Similarly, DejaVu also depends heavily on frequent CPU-to-GPU data transfers, facing the same bandwidth bottlenecks over PCIe that restrict overall throughput.

GPU–CPU Hybrid Execution
Another class of solutions distributes computation between both GPU and CPU rather than using the CPU solely as a memory extension. A notable example is *llama.cpp*, which achieves functional inference on consumer devices with relatively low-end hardware. While this improves accessibility, its performance remains significantly behind data center GPUs. For instance, *llama.cpp* reports around 600 ms latency compared to approximately 45 ms on an NVIDIA A100.

Solution Overview

The researchers observed that neuron activation in large language models follows a skewed power-law distribution during inference. This means that a small subset of neurons (~<20%) contribute to more than 80% of the total activations. This pattern largely arises from the sparsity induced by ReLU activation functions, where many neurons remain inactive for a given input.



PowerInfer leverages this insight by exploiting the inherent *locality* in LLM inference. Instead of treating all neurons uniformly, it dynamically allocates computation based on activation frequency: the small set of “hot” neurons, which are frequently activated, are placed on the GPU, while the “cold” neurons, which are rarely used, are managed by the CPU. This design minimizes unnecessary data transfer and better utilizes available hardware resources.

The researchers also note that modern CPUs include support for SIMD (Single Instruction, Multiple Data) operations. Most contemporary processors implement at least the AVX2 instruction set, enabling parallel arithmetic computations on multiple values within a single cycle.

This allows the CPU to efficiently handle the cold neurons without becoming a significant computational bottleneck.

Limitations

Limited Activation Function Support.
Finally, PowerInfer currently works only with models that use ReLU activations. Since most recent LLM architectures have shifted toward alternative activation functions like SwiGLU or GELU, this restriction limits PowerInfer's compatibility with modern models.

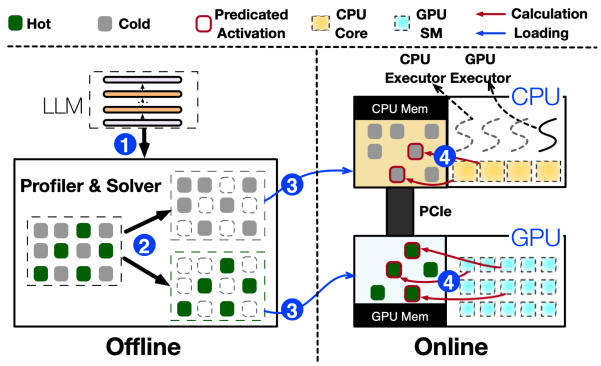
Reduced Benefits for Low-Sparsity Models
PowerInfer's performance gains rely heavily on the degree of activation sparsity. Models using ReLU activations, which typically exhibit over 90% sparsity, achieve significant speedups of 8–11×. In contrast, models based on SwiGLU activations show much lower sparsity (around 50%), resulting in only modest improvements of 1.5–1.7×.

Table 8. Generation speed (tokens/s) comparison for SwiGLU and ReLU-based LLMs.

Setting	Model	PowerInfer	llama.cpp	Speedup
PC-High	Yi(SwiGLU)-34B	1.7	1.0	1.7×
PC-Low	LLaMA(SwiGLU)-2-13B	3.1	2.1	1.5×
PC-High	OPT-30B	12.0	1.1	10.9×

High Setup Overhead.
Each new model demands considerable preprocessing effort, including manual profiling to measure neuron activation frequencies, solving integer linear programs (ILP) for optimal partitioning, and training activation predictors to estimate which neurons will activate during inference.

Compiler Challenges



Current state-of-the-art sparsity-aware compilers are not well-suited for PowerInfer's neuron-level execution pattern. The system requires extensive customization and optimization for each model, which introduces substantial engineering overhead.

Future Research Directions

Optimizing non-RELU models, a more diverse spectrum of models:

For models that use different activation functions, coming up with alternative sparsity approximation methods to reduce inference latency.

Integration with Speculative Inference

The paper notes that "integrating speculative inference into PowerInfer could further boost LLM inference speed." Using smaller, faster models with similar sparsity techniques could result in much faster inference speed while maintaining high model performance.

Questions

Q: What happens when neurons are incorrectly predicted? (predictor has 93% accuracy)
A: Slight performance drop, but that drop is reported to be trivial. The performance gains outweigh that loss. Also, the neurons that were inaccurate were of lesser importance.

Q: Do you think it would be valuable to improve the predictor model?
A: No, it would result in diminishing returns.

Q: A small portion of neurons are activated for MLP layers. Do the authors mention what the proportion is for the sub-attention layers?
A: They say that half of the sub-attention layers are activated, but don't really focus on it.

Q: This approach works well for ReLU models that are sparse. Do the authors mention if it can work better with SwiGLU?
A: The authors implemented their idea of SwiGLU, and were able to achieve some performance increase, but not as much.

Q: Can anything be done to make the model work better with SwiGLU?
A: It is a more challenging task to work with SwiGLU, as the activation differs greatly. A potential solution could be to work on improving the predictor accuracy. Very few models are still using ReLU - People are moving on to SwiGLU because people are looking for ways to increase parameters, and SwiGLU provides the ability to do that.

Q: What are the main differences between PowerInfer and Flexgen?

A: PowerInfer focuses on transferring the activation between the hardware while flexgen transfers the layer waste itself.

Q: Why was the target/solution of both research groups different?
A: The two papers had different objectives. PowerInfer prioritized low latency and efficient inference on small batches using CPU–GPU collaboration. In contrast, FlexGen focused on maximizing throughput for large batch inference by optimizing data movement and scheduling across devices.

Q: Do you think it is possible to only offload the attention to the CPU, so that you could operate on higher batch sizes?
A: There is computation delegation where computing attention on the CPU does result in I/O optimizations.

Summary of “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU”

Problem and Motivation

Large language models contain billions of parameters, requiring massive memory and computational resources to run efficiently. This makes them inaccessible to users without high-end data center GPUs. The primary motivation behind FlexGen is to make large-scale model inference feasible on more modest hardware by reducing memory demands and improving efficiency.

The overarching goal is to enable large model inference on limited-memory devices through techniques such as model compression and efficient memory management. Smaller model footprints translate directly to lower memory requirements and broader accessibility.

Related Works

Model Compression
Prior work on model compression uses techniques such as quantization and pruning to reduce model size and memory consumption. While effective, these methods can degrade accuracy and require retraining or fine-tuning.

Collaborative Inference
Another line of work focuses on distributed or decentralized inference, where computation is shared across multiple devices. This approach helps scale workloads but adds complexity in coordination and data transfer.

Offloading
Frameworks like DeepSpeed ZeRO-Inference and Hugging Face Accelerate implement offloading strategies that move parts of the model between GPU, CPU, and disk to fit large models on limited-memory systems. However, these methods often suffer from high latency due to slow data movement over PCIe or storage interfaces.

Solution Overview

FlexGen focuses on optimizing offloading strategies to efficiently run large language model inference on limited-memory hardware. The key challenge lies in determining the optimal order for traversing layers and tokens during batched inference, which becomes effectively a graph traversal problem.

Traversal Strategies.
Traditional systems use row-by-row traversal, which processes each token sequentially across layers. This allows the key-value (KV) cache to be freed immediately after completing a row, reducing memory pressure. However, this approach requires loading new weights for each layer repeatedly, resulting in significant I/O overhead.

In contrast, column-by-column traversal processes all tokens for a single layer before moving to the next. This enables weight reuse within a column, but it limits the ability to free KV cache early.

ZigZag Schedule.
FlexGen introduces a ZigZag scheduling strategy that combines the advantages of both methods. It minimizes redundant weight loading by reusing weights across tokens within a column while maintaining enough flexibility to release KV cache efficiently. This hybrid traversal significantly reduces I/O between the GPU, CPU, and storage devices.

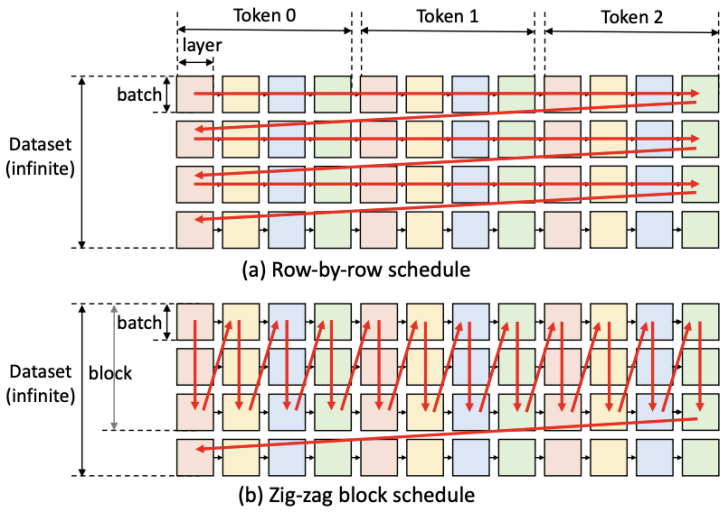


Figure 3. Two different schedules. The red arrows denote the computation order.

Tensor Placement.
FlexGen assigns tensors to different memory levels based on their access patterns. Weights are managed at layer granularity, while activations and KV caches are handled at tensor granularity to optimize data movement and memory usage.

Cost Model.
The system models total inference latency as the sum of prefill and decode times, both of which are heavily influenced by I/O bandwidth between CPU, disk, and GPU rather than pure computation.

Limitations

FlexGen's performance is largely I/O-bound, especially during the decoding phase, where data transfer dominates runtime. The policy search that determines the optimal scheduling strategy can also require manual fine-tuning to achieve the best performance.

While FlexGen demonstrates strong throughput gains, its latency–throughput tradeoff becomes less favorable when using multiple GPUs. Additionally, the experiments were conducted using 208 GB of system RAM, far exceeding the capacity of typical consumer hardware.

Another limitation is inefficiency with variable-length input sequences. Because FlexGen pads all inputs to the same length for batching, it spends unnecessary time processing padded tokens, reducing efficiency for uneven sequence distributions.

Future Research Directions

Extending Offloading to Multi-GPU Settings: Current FlexGen strategies achieve near-optimal performance on a single GPU but still face I/O bottlenecks and low GPU utilization. Future research could explore multi-GPU offloading strategies, using model parallelism (tensor parallelism for lower latency, pipeline parallelism for higher throughput) to reduce per-GPU memory pressure and potentially achieve super-linear scaling in decoding performance

Integrating Approximate Methods for Higher Throughput: The paper shows preliminary success with 4-bit group-wise quantization (for both weights and KV cache) and sparse attention to cut memory and I/O costs while maintaining accuracy. Future work could develop more advanced or hybrid approximation methods to further reduce resource requirements and improve throughput-accuracy tradeoffs, making FlexGen more efficient on commodity hardware

Questions

Q: Why was hugging face used as a method of evaluation and was there a better method?
A: Not really any offloading models, so best thing they could compare to ILP output can result in out of memory for the hardware component.

Q: What is put in place to detect this?
A: The best approach right now is just to refer to the peak memory usage, however it has limitations.

Summary of Class Discussion

Q: **If we have AI in our computers and each different application is using edge AI to run their processes, how do all these applications coexist? (Assuming we have enough resources)**

The class discussed how multiple AI applications could coexist on a single computer if each one relied on edge AI for local inference. In theory, each LLM instance could have its own local IP address, allowing different applications to communicate independently through networking protocols such as TCP or UDP. However, this raises a major challenge, as most large models cannot fully fit within a consumer GPU's memory.

It was suggested that managing such models should not be the responsibility of the operating system, but rather handled through networking-level communication between applications. For users who prefer privacy or wish to use closed models, one solution is to store the model locally so that it is accessible only to the owner, avoiding exposure to external servers. Relying on cloud-based inference for every request can quickly become cost-prohibitive, especially at scale.

Another discussion that spins off this is how can we use closed source models, in an optimized fashion, locally. Most common consumer-grade hardware engines like vLLM or Ollama require knowledge of the model architecture and weights. Could we engineer solutions that are antagonistic to the model format and structure itself? We could imagine companies like Microsoft could offer running small, closed source models on device, that could take on certain user requests rather than sending to the server. This would result in lower inference costs, time, and energy saved (potentially cheaper user cost to use AI services). Designing engines for closed source models is another area of innovation under explored, but with high potential.

Another point raised was the imbalance between the complexity of queries and the computational effort required to generate responses. This led to the idea of a filtering system that could detect “easy” queries and route them to lightweight local models, while sending more complex ones to more powerful inference engines. Finally, an OS-like resource manager was also proposed. One that could allocate GPU and CPU resources fairly among multiple models running simultaneously.